



CONTEXT ENGINEERING

PARTICIPANT GUIDE

** INTERNAL USE ONLY **

TABLE OF CONTENTS

OUTCOMES.....	3
OPENING ACTIVITY: DIGIMON BATTLE.....	3
WHAT IS CONTEXT?.....	3
PROMPT VS. CONTEXT ENGINEERING.....	3
CONTEXT SOURCES.....	4
CONTEXT ENGINEERING.....	4
ACTIVITY: CONTEXT SOURCE HUNT	5
REVISIT.....	5
CONTEXT BLOAT	6
CONTEXT ROT	6
SLOWER RESPONSES.....	6
DEGRADED FOCUS.....	6
STALE BEHAVIOR.....	7
SPOTTING CONTEXT ROT	7
ACTIVITY: DIAGNOSE CONTEXT ROT	7
SCENARIO 1	8
SCENARIO 2	8
SCENARIO 3	9
REVISIT.....	9
CONTEXT ENGINEERING STRATEGY	10
SELECT.....	10
STRUCTURE.....	10
CLEAN UP	11
WHEN TO START FRESH.....	11
ACTIVITY: CONTEXT ENGINEERING.....	12
REVISIT: KEEP, REMOVE, OR SUMMARIZE.....	13
ACTIVITY: CREATING A CONTEXT PLAN	13
ADDITIONAL RESOURCES	14

CONTEXT ENGINEERING

OVERVIEW

In this course, you will explore how GitHub Copilot responds to more than just the words in your prompt. GitHub Copilot sees a full set of information called context, including open files, system instructions, conversation history, tools, and examples. When that context is bloated, outdated, conflicting, or irrelevant, Copilot output can become slower, less accurate, or harder to trust. Through hands-on activities and guided discussion, you will build a context engineering mindset to help Copilot stay focused, reduce unnecessary noise, and produce outputs that are easier to review and apply in your work.

OUTCOMES

By the end of this course, you will have completed the following:

- Identify 4 sources of context that can influence output
- Explain 3 effects of poor context engineering on AI-generated responses
- Create a prompt that uses 3 context engineering decisions to support a coding task

OPENING ACTIVITY: DIGIMON BATTLE

Open the Excel file provided by your trainer, and find the best Digimon to defeat your opponent in the two rounds.

WHAT IS CONTEXT?

Context is everything the model can see when it generates a response. This includes your prompt and all of the surrounding information. In GitHub Copilot, context can include open files, related files, system instructions, conversation history, tools, and other available information.

It is important to know that context changes after each response. When Copilot gives you a response, it is not only responding to your prompt — it is responding to your prompt plus all currently available information.

PROMPT VS. CONTEXT ENGINEERING

Prompt engineering and context engineering both work together to shape Copilot output, but in different ways.

- Prompt Engineering: Focuses on the immediate prompt only. It shapes what you ask. For example, being clear about the task, constraints, and expected output formats.
- Context Engineering: Looks at the larger environment. It asks: what can the AI see right now? Is that information useful, current, and relevant to this task?

CONTEXT SOURCES

Everything GitHub Copilot can see is considered context, including the following categories:



*** INTERNAL USE ONLY ***

CONTEXT ENGINEERING

Context engineering is the art and science of curating the optimal set of tokens for an LLM at inference time. It is the evolution beyond prompt engineering and the foundation of effective agent design. Some important context engineering concepts include:

- More context is not always better
- Copilot cannot always differentiate good vs bad or new vs old
- Context needs continuous cleaning

ACTIVITY: CONTEXT SOURCE HUNT

Open the sample repo provided by your trainer, also linked below. Explore the repository and find 4 different sources of context. For each source you find, record it in the table below and mark whether you think it would be helpful, harmful, or neutral for a Copilot response. <https://code.uwm.com/projects/BT/repos/ce-context-source-hunt/browse>

Tip: Use the context sources on the previous page to help guide your search.

Context Source	Helpful / Harmful	Why?

INTERNAL USE ONLY

REVISIT

Which source of context do you think influences Copilot output the most in your role, and why?

LACK OF CONTEXT

Lack of context happens when important details are missing from what Copilot can see, such as the current business rule, existing test patterns, testing guidelines, naming conventions, or examples of what good output should look like.

When Copilot does not have enough context, it will usually still respond, but it may fill in the gaps with assumptions. This can lead to output that is generic, incomplete, misaligned to the task, or even hallucinated.

CONTEXT BLOAT

Context bloat occurs when too much information is added to the model's context, especially when it's not relevant to the current task. This can include:

- Too many open files in the workspace
- Long conversation history with outdated decisions
- Repetitive or low-value examples
- Overlapping tools or MCP servers

Context bloat doesn't necessarily mean the context is bad on its own, but that there's too much and it is not focused on the current step.

CONTEXT SWEET SPOT

The context sweet spot is the middle point between too little context and too much context. Copilot should have enough information to understand the current task, but not so much that important details get buried.

Good context includes relevant requirements, clear patterns, useful examples, and enough detail to guide the model. It also removes unnecessary noise so Copilot can stay focused on the current task.

CONTEXT ROT

Context rot happens when growing or messy context makes the AI less precise over time. The model may still respond confidently, but the output can drift, miss important details, or rely on outdated information. When there is too much context, the model becomes slower, less accurate, and more expensive. Context rot leads to the following negative effects:

SLOWER RESPONSES

The AI must process more tokens per request, meaning it has more input to review before generating a response. This slows down response time and increases the cost of the interaction. Context bloat is a common and avoidable cause of slow Copilot responses.

DEGRADED FOCUS

Key details get buried under too much information. Copilot might overlook relevant details or give too much weight to less relevant ones. This is when output becomes generic, misaligned with the current task, or simply off target.

STALE BEHAVIOR

Old decisions remain in the context and continue shaping responses even after the task or direction has changed. For example, if you earlier asked Copilot to use one implementation approach and then changed direction, Copilot may keep referencing the original approach because it is still in the conversation history.

Notes:

SPOTTING CONTEXT ROT

Context rot is not always obvious, even when the output looks confident. Watch for the following warning signs:

- Inconsistent variables: The AI drops or alters naming conventions, function names, or code structures that were previously established.
- Repeated approaches: The model repeatedly suggests a solution or code block you already dismissed or implemented.
- Vague or hallucinated responses: The model's reasoning becomes shallow, producing generic patterns or plausible but incorrect information.
- AI contradicting itself: The model conflicts with its own earlier technical choices, rules, or design decisions made just a few prompts prior.

ACTIVITY: DIAGNOSE CONTEXT ROT

On the next pages, for each scenario, answer the two questions in the space provided below.



SCENARIO 1

A developer asks Copilot to write a unit test for a loan calculation method. The context includes the full solution file, old deprecated services, UI components unrelated to the logic, and a large markdown document with general coding standards.

What's wrong with the context?	
What effect could it have on Copilot's output?	

SCENARIO 2

A developer asks Copilot to write a unit test for a method. The context includes only the method itself — no testing guidelines or examples.

What's wrong with the context?	
What effect could it have on Copilot's output?	

INTERNAL USE ONLY

SCENARIO 3

A developer asks Copilot to refactor a service method. The context includes a "best practices" document and an older service file that uses a completely different set of patterns.

What's wrong with the context?	
What effect could it have on Copilot's output?	

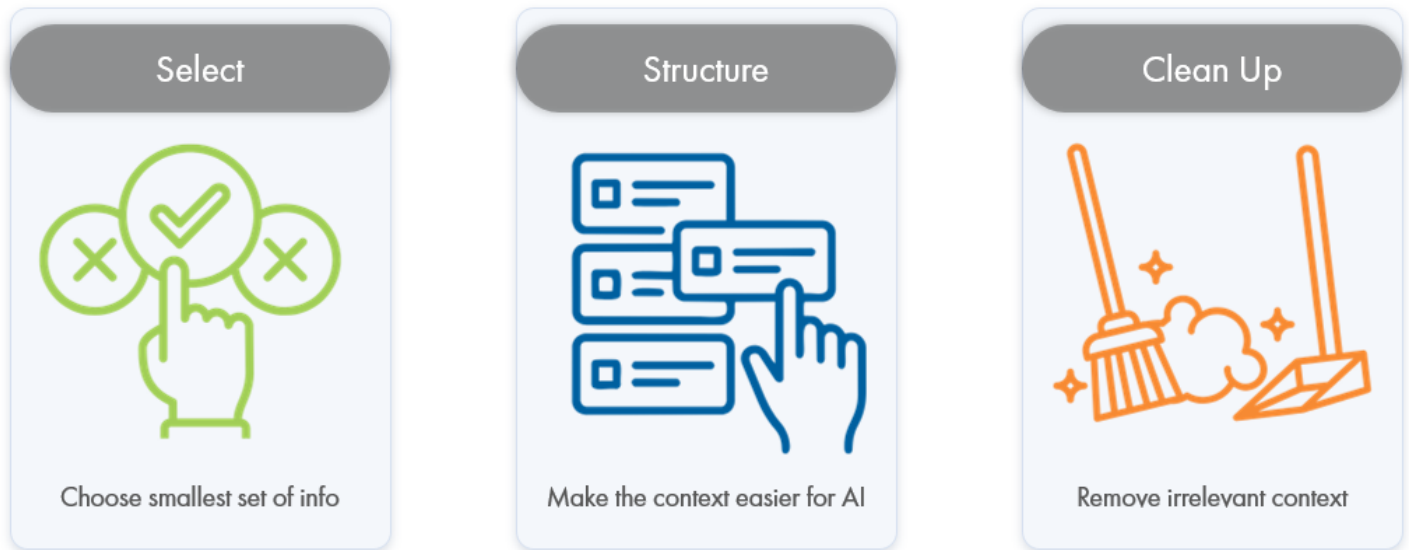
INTERNAL USE ONLY

REVISIT

Why is more context not always better for AI?

CONTEXT ENGINEERING STRATEGY

Context engineering is about giving Copilot the right information at the right time. You can use this three-part strategy before asking Copilot to complete a large task.



SELECT

Choose the smallest set of relevant information Copilot needs for the current step. Not every related idea, just what matters for this specific request.

- Select only the files relevant to the current task
- Include the current task requirements
- Keep necessary examples that directly apply
- Exclude unrelated noise

STRUCTURE

Make the context easier for Copilot to understand and use. Even useful information can become hard to use if it is disorganized or unclear.

- Organize instructions with headings or clear sections
- Provide a short checklist or a clear goal statement
- Keep examples clearly labeled and easy to identify
- Separate the current goal from background or reference information

CLEAN UP

Remove or reduce context that is no longer helping. Stale history, old decisions, unrelated files, and incorrect assumptions all add noise and can lead to context rot.

- Close unrelated files in the workspace
- Summarize or compress long conversation history
- Remove outdated instructions or old examples
- Start fresh when the current context is working against you

WHEN TO START FRESH

Sometimes the best context engineering decision is to start a new session with clean context. Watch for these signs that your current context is doing more damage than good:

- You keep re-explaining the same issue to Copilot
- Copilot is confidently giving an answer you know is incorrect
- An old approach keeps resurfacing even after you changed direction
- The conversation is getting into strange loops or circular reasoning

When switching to a new topic, starting a new Copilot session with fresh context is best practice.

Notes:



ACTIVITY: CONTEXT ENGINEERING

Looking at the sample repo, write a context brief you could use including the 3 context engineering decisions to improve the context window.

<https://code.uwm.com/projects/BT/repos/ce-context-engineering-practice/browse>

Your brief:

Select:

Structure:

Clean up:

REVISIT: KEEP, REMOVE, OR SUMMARIZE

For each context item below, check the box that best describes what you would do with it before your next task.

Context Item	Keep	Remove	Summarize
Current task requirements	☐	☐	☐
Chat history from a previous troubleshooting session	☐	☐	☐
Unrelated files open in the workspace	☐	☐	☐
Old tool results	☐	☐	☐
A system instructions file scoped to this project	☐	☐	☐

NOTES:

CLOSING ACTIVITY: CREATING A CONTEXT PLAN

Think about a recent session or your day-to-day use of GitHub Copilot and complete the four sentences below.

"The next time I use GitHub Copilot or an agent, I will improve context by..."

Keeping:
Removing:
Summarizing:
Starting fresh when:

** INTERNAL USE ONLY **

ADDITIONAL RESOURCES

- Context Engineering KB Page - <https://kb.uwm.com/x/JFVaX>
- Understanding Token Costs KB Page - <https://kb.uwm.com/x/Y-SFX>

**If you have any questions or concerns,
please reach out to ITTraining@uwm.com**